Year Up United
Data Analyst Training Academy: Beginning Python

Week 5 Lab Workbook

This courseware is to be distributed to accompany instructor led training.

Copyright © 2026 Pluralsight & The Reliable Collaboration Company

Table of Contents

MODULE 1 INTRODUCTION TO PYTHON 3

EXERCISE 1.A GETTING READY FOR WEEK 5 4
EXERCISE 1.B GETTING STARTED WITH PYTHON 13

MODULE 2 BUILDING EXPRESSIONS 17

EXERCISE 2.A MAKE CALCULATIONS WITH PYTHON 18
EXERCISE 2.B MORE PRACTICAL MATH SCRIPTS 23

MODULE 3 WORKING WITH DATA TYPES 25

EXERCISE 3.A NUMERIC CONVERSION EXERCISES 26
EXERCISE 3.B CREATING COLLECTIONS 28

MODULE 4 CONDITIONAL LOGIC & FLOW CONTROL 31

EXERCISE 4.A USING CONDITIONALS 32
EXERCISE 4.B LOOKUPS AND GREETINGS 34
EXERCISE 4.C LOOP DE LOOP 38
EXERCISE 4.D BONUS PROJECT: NUMBER GUESSER 41

Module 1

Introduction to Python

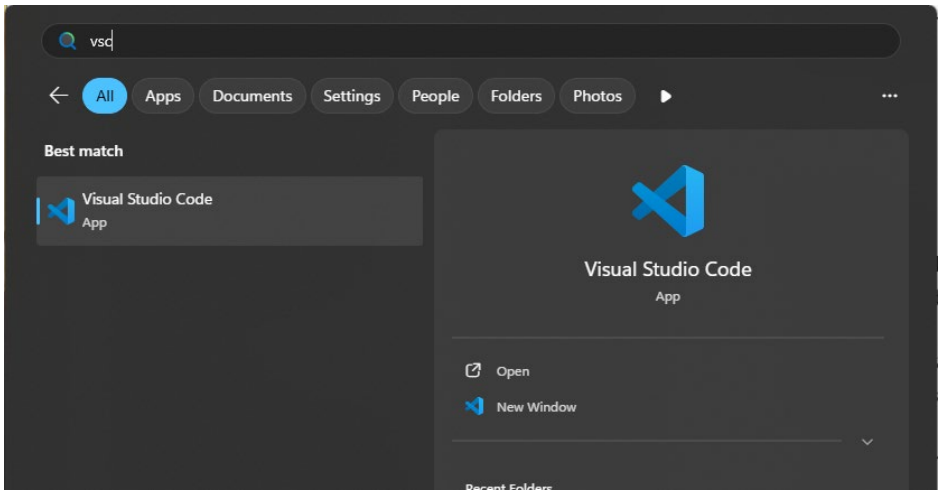
Exercise 1.A Getting Ready for Week 5

Getting Ready

This week you will continue to use git and GitHub to save your lab work, saving all homework to DataAnalytics/week-05. You will also begin working in Visual Studio Code.

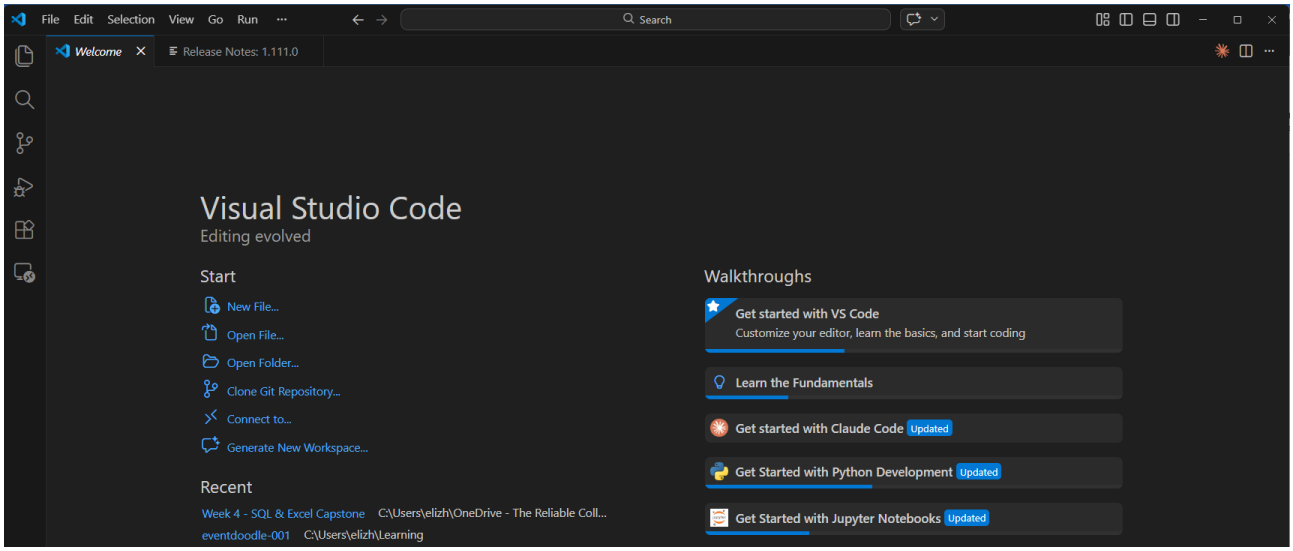
First confirm that you have Visual Studio Code installed and ready to go.

1. From your computer's Start menu, open Visual Studio Code. (You should be able to find it by typing "visual" or "vsc" into the search bar.)

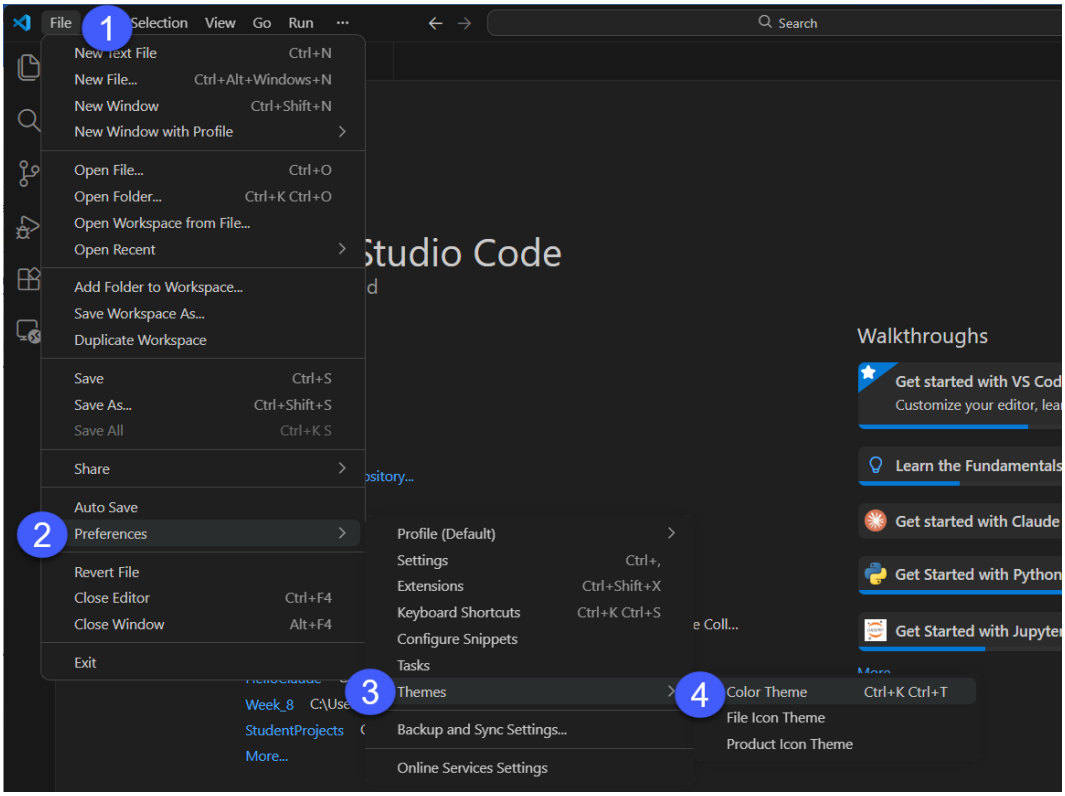


2. Once you have Visual Studio Code open, take a moment to get oriented.

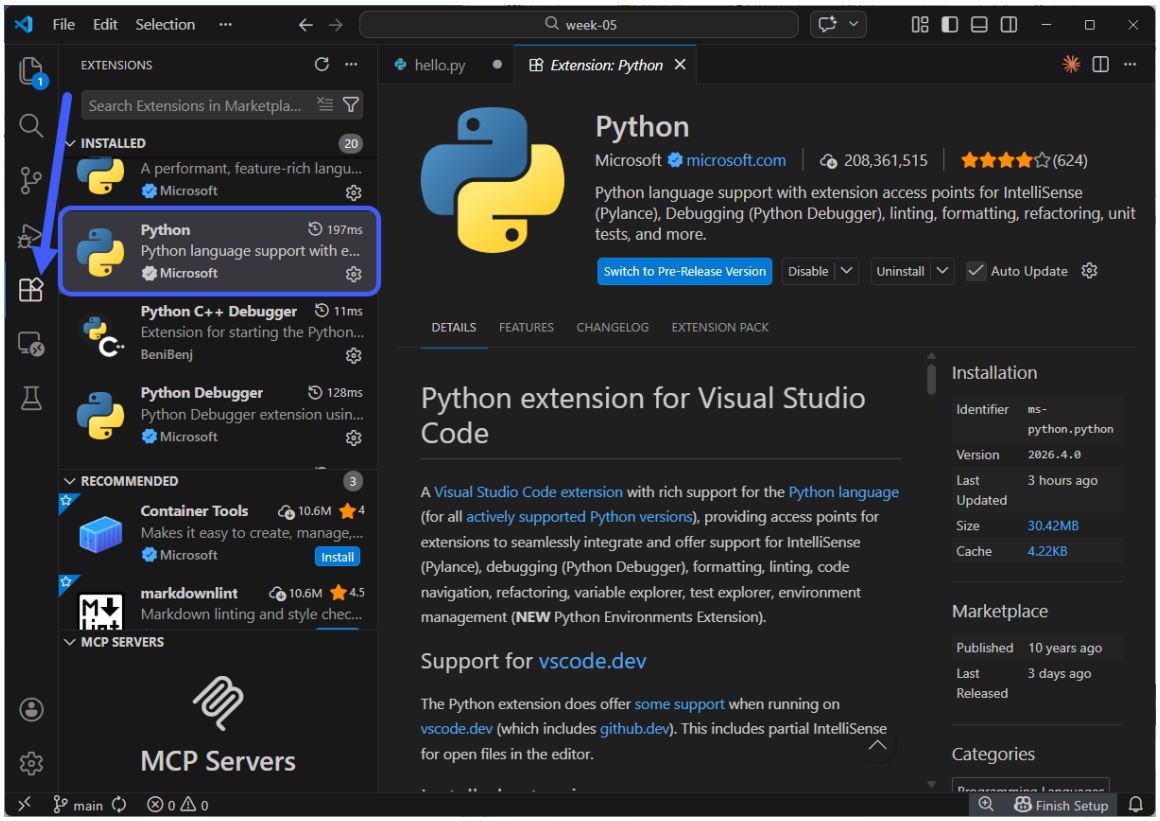
   a) VS Code may open to a Welcome screen, or notes from the latest release, or to the most recent project that you had open. Note the tabs at the top for open files.



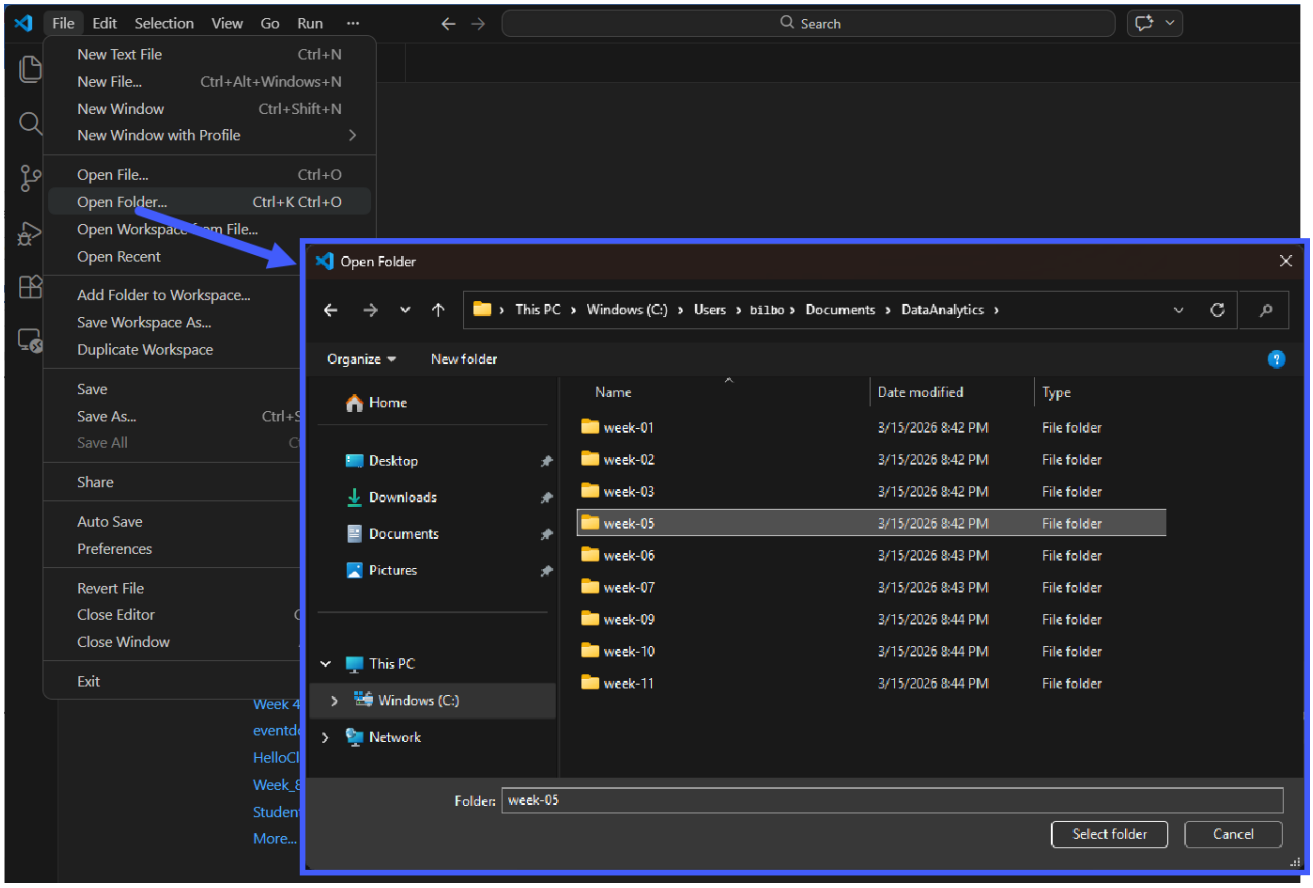
b) If you want to change the color scheme that VS Code uses, you can do so from the **File** menu > **Preferences** > **Themes** > **Color Theme** (shortcut Ctrl+K Ctrl+T)



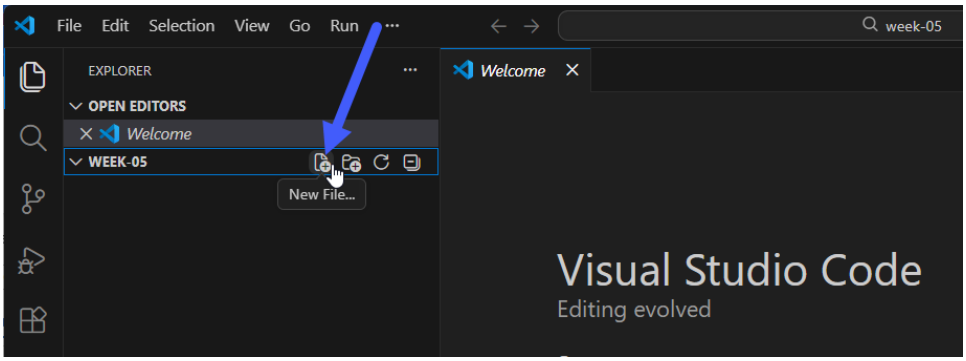
3. On the left side, click Extensions (the icon that looks like four squares) and confirm that you have the Python extension installed.



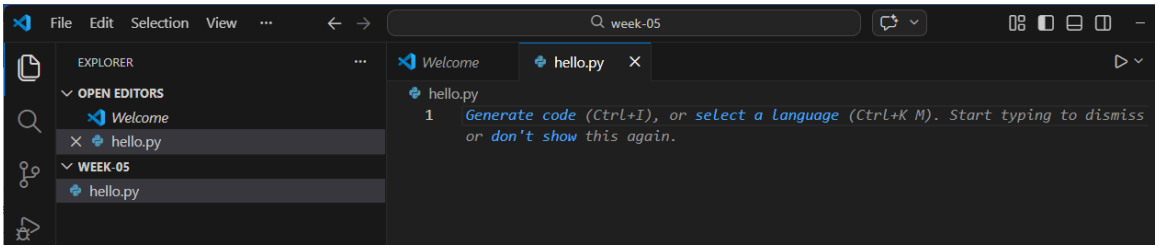
4. Use **File** > **Open Folder** to open to **Documents/DataAnalytics/week-05**



a) The folder should open in a panel on the left hand side of VS Code. Since you haven't created any files yet, it's probably empty. Mouse over the left panel then select the New File icon to create a new blank file.



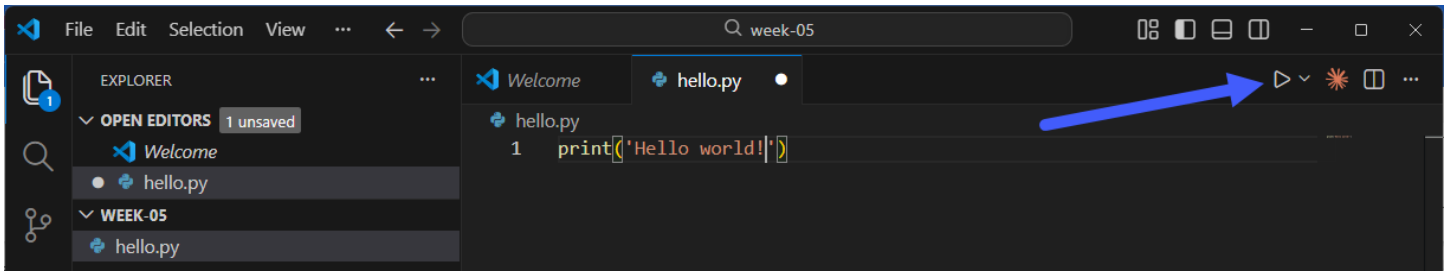
b) Name the file **Ex1A_hello.py** and hit enter.



First program:

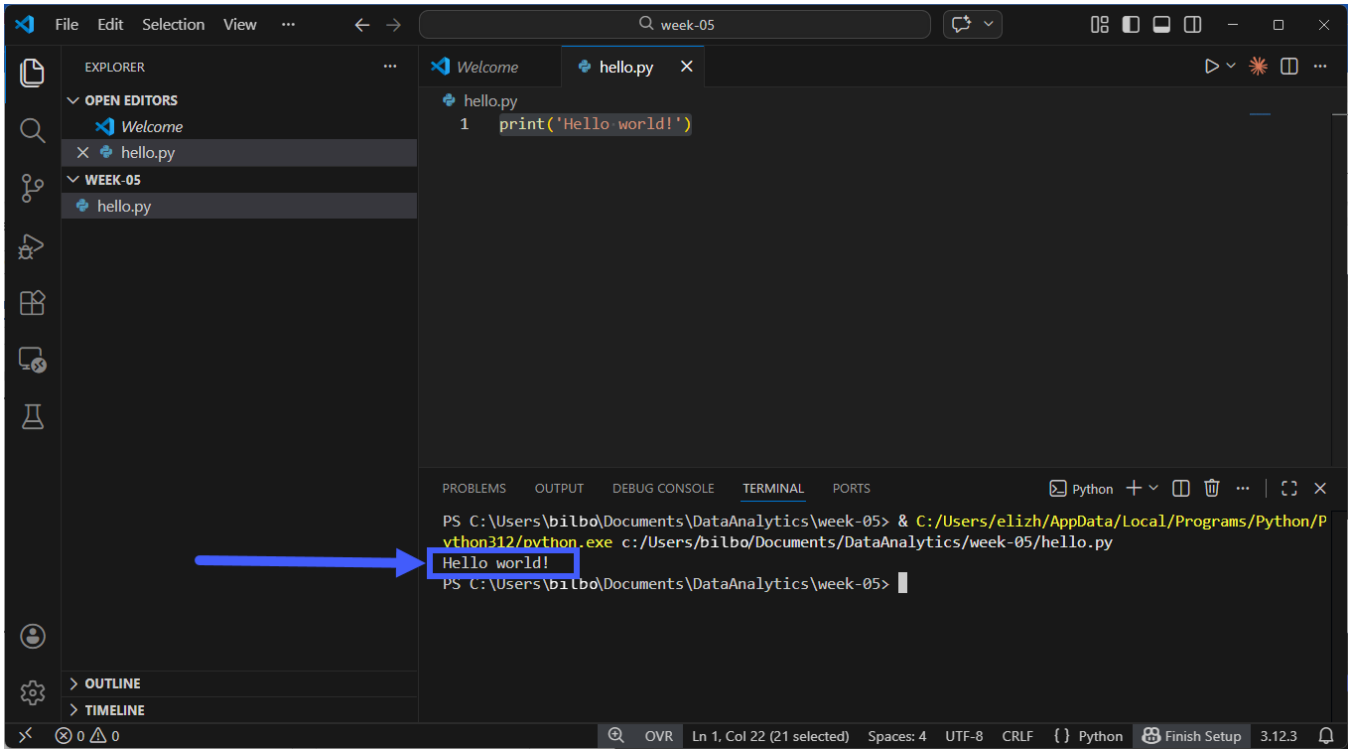
5. At line 1 in Ex1A_hello.py, write your first program:
   `print('Hello world!')`

   Then click the Run icon (▷) in the upper right corner



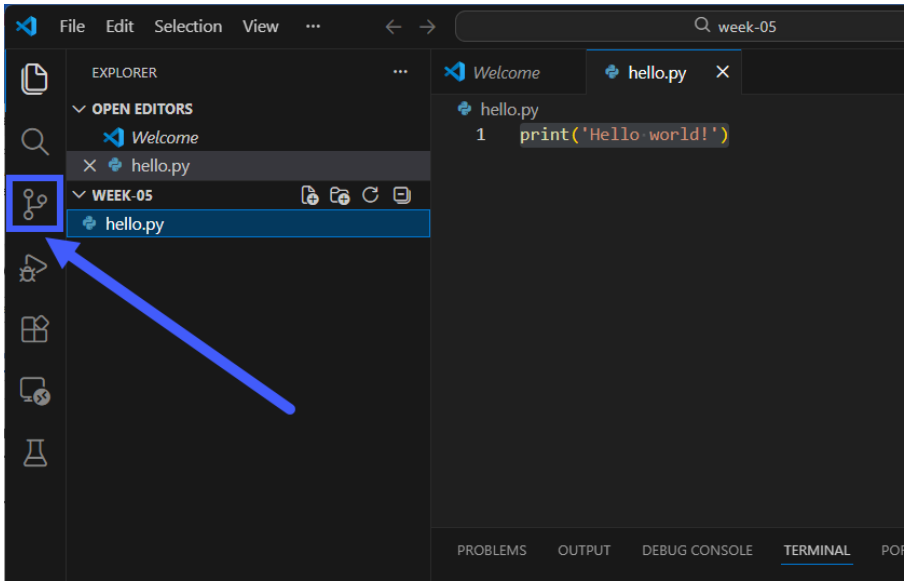
Note: You may notice as you type that different pop-up messages with help text appear. This feature will become very useful as you start to get more familiar with coding and with what the messages are hinting about, but if you want to dismiss a help pop-up, you can hit Esc on your keyboard to close it.

6. Your program output will display in the terminal that pops up in VS Code.

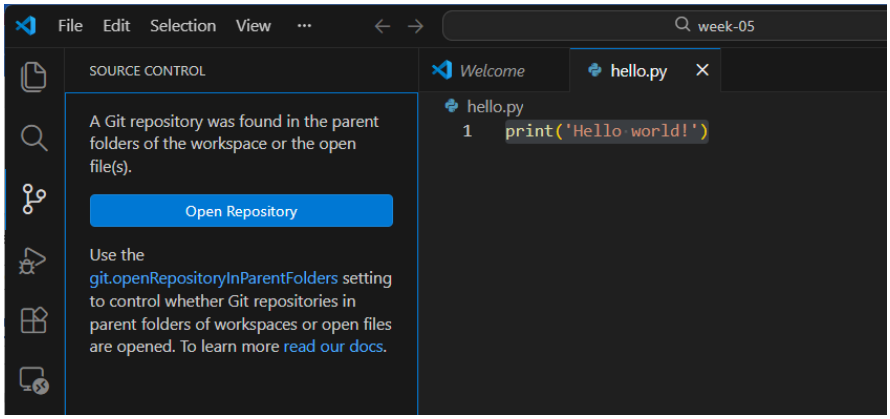


Congratulations, you're a Python programmer!

7. Save your changes to Ex1A_hello.py, then click on the Source Control icon on the left (looks like three dots connected by two branched lines).

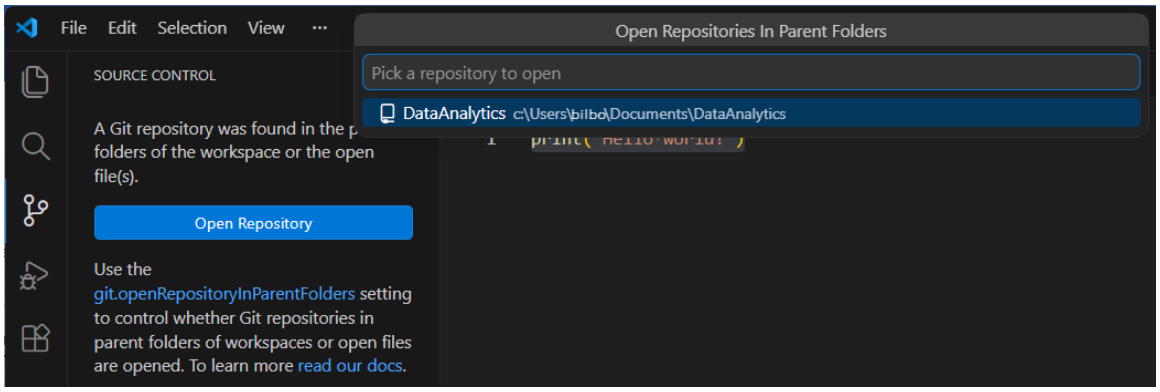


8. Because you are working in DataAnalytics/week-05, VS Code should recognize that there is a repository attached to this folder. Click the big button for Open Repository:

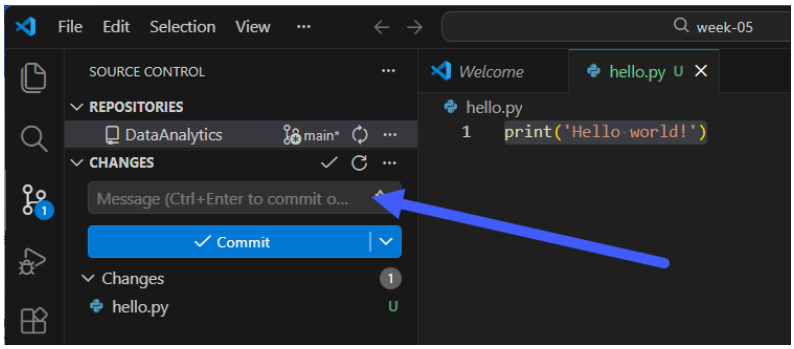


If instead VS Code tells you that no repository is found, you may need to do some troubleshooting to figure out why your DataAnalytics repository is not working as expected. Note: you can also follow the workflow of manually pushing updates to DataAnalytics/week-05 using Git Bash as you've done in previous weeks

9. Once you click Open Repository, select the DataAnalytics repository from the drop down menu that appears at the top:

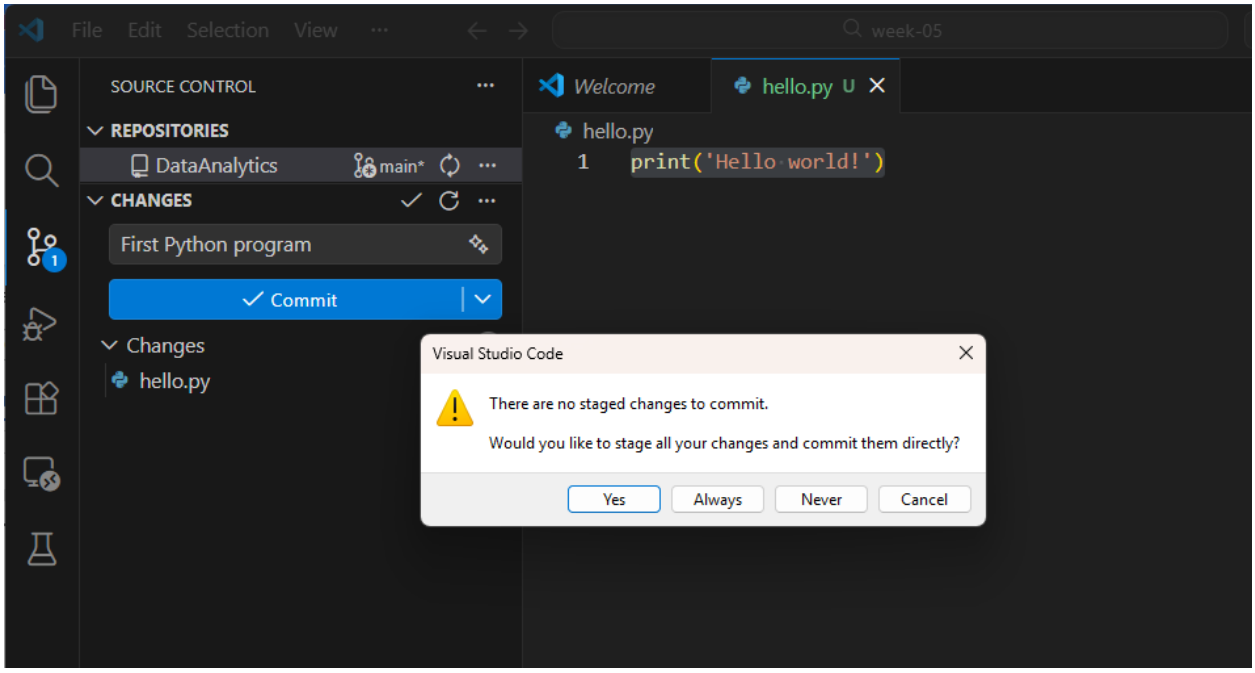


10. The left panel should update to reflect the repository structure. In the text box, enter the commit message *first Python file*, then hit Commit.

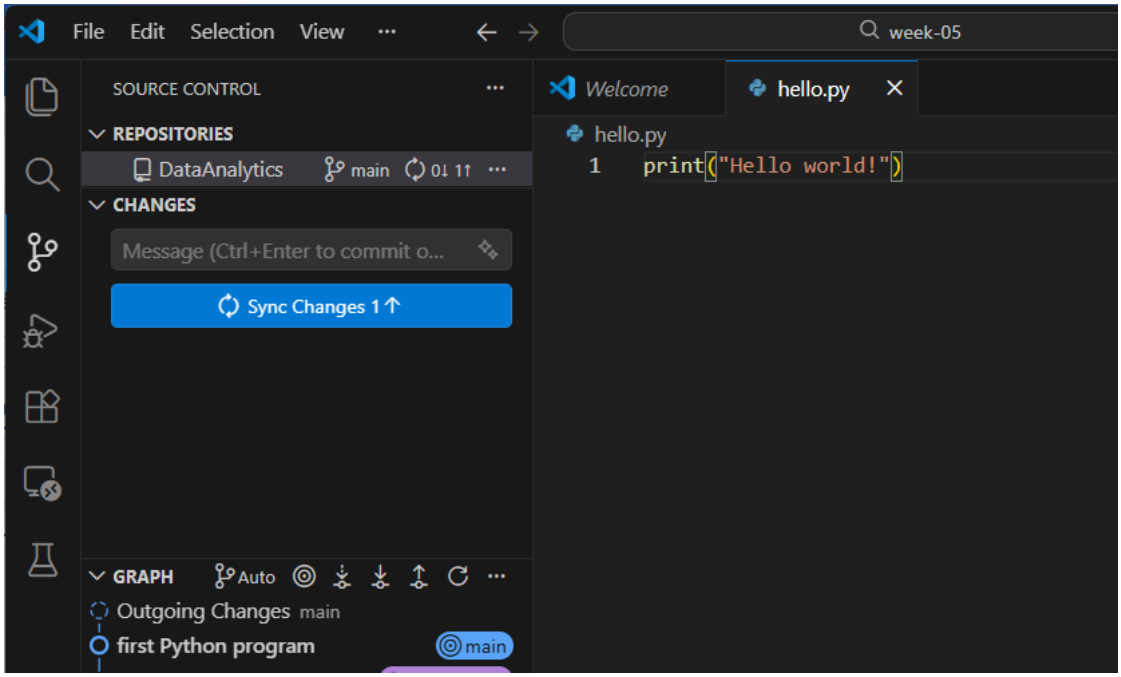


(Note: the commit message here is equivalent to what you used to type in Git Bash in quotation marks after the command `git commit -m`. All changed files will be tracked under the heading Changes, so you can see exactly what's going into your commit.)

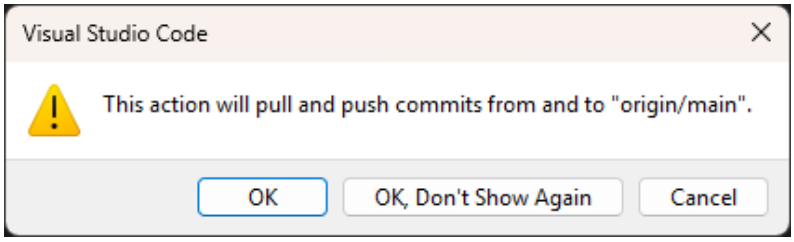
11. VS Code will pop up a message asking if you would like to stage changes to commit. Click Yes to continue.



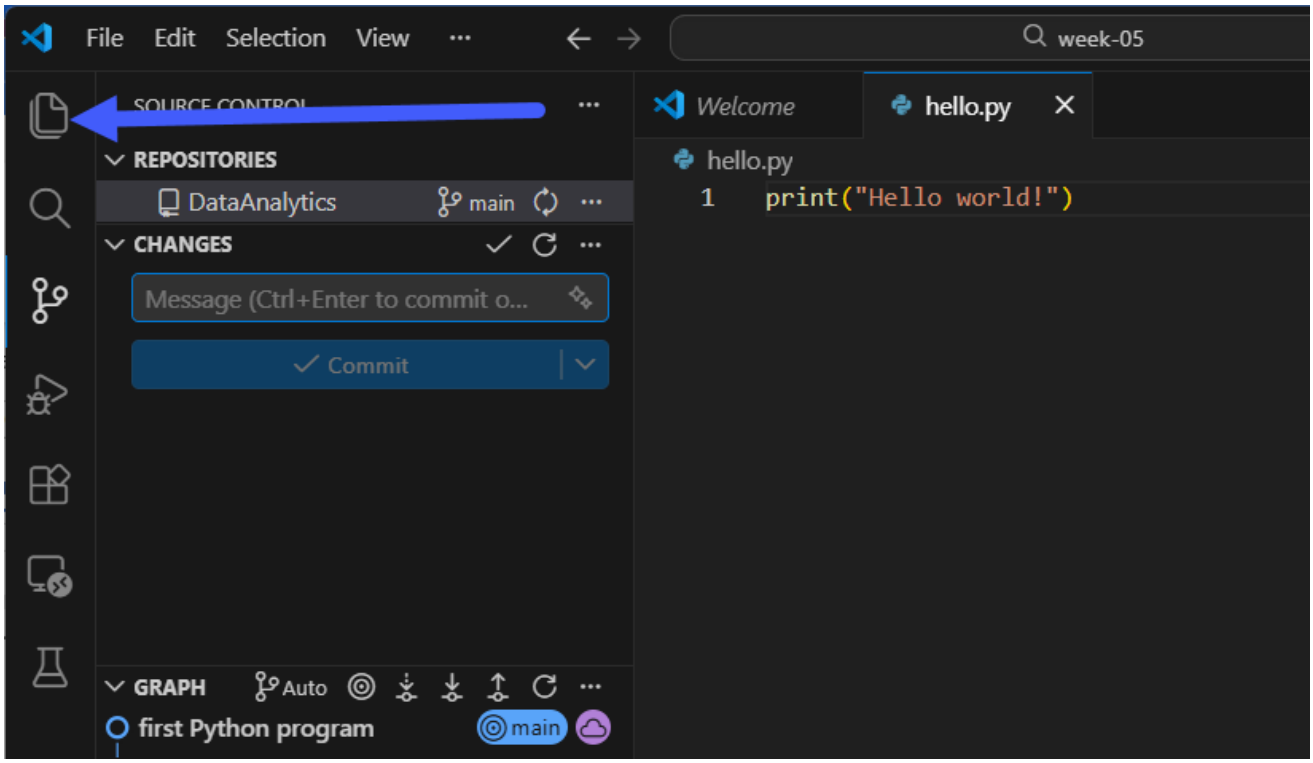
12. Then click Sync to publish your changes to GitHub.



If you get a pop-up to confirm this will push/pull changes, you can either click “OK” or “OK, Don’t Show Again.” (The latter option means you will not get that pop-up again when you sync future changes.)



13. When your changes have finished syncing, click the Explorer icon at the upper left (looks like a stack of pages) to return to the week-05 folder structure.



Practice working with variables:

14. Keep working in Ex1A_hello.py. Hit enter a couple of times after your first line of code.

15. Create a variable named `message` to contain the string 'Hello world!', then use the variable to print `message`.

```
3  message = 'Hello world!'
4  print(message)
```

16. When you click the Run button at the top right, you should see the message "Hello world!" printed twice in the terminal. Why is it printing twice? Write the answer as a

comment.

```
print("Hello world!")

message = 'Hello world!'
print(message)

# "Hello world" prints twice because ...
```

17. Hit enter a few more times to leave a blank line, and then add another comment :
    `# Displaying dollars and cents`

18. Create two variables for `dollars` and `cents`:

    a) `dollars` should contain a value of 3

    b) `cents` should contain a value .50

    c) Then add them together inside a print statement:
    `print(dollars + cents)`

19. What do you notice about the result? Add a comment with your observation.

20. Add a new line below the print statement to update the value of cents by adding .25 to it, like this:
    `cents = cents + .25`
    Then print dollars + cents again to see the new result.

21. Using what you have learned so far, create a variable called `d_str` for the string `'3 dollars'` and another variable called `c_str` for the string `'50 cents'`, then combine them in a print statement that yields the text output *3 dollars 50 cents* (Hint: you will need to figure out how to add a blank space in the middle using `" "`.)

22. When you are done, remember to save changes to Ex1A_hello.py and use the commit/sync workflow in VS Code to push your changes.

Exercise 1.B Getting started with Python

Lab 1

1. In Visual Studio Code, within your **week-05** folder, create a new file named **Ex1B_variable_examples.py**

2. In Python, variable names *must* start with a letter or underscore (not a number) and are conventionally written in lower case with underscores in place of spaces (snake_case). Create example variables with good names to represent the following items (also make up a sample value for each variable to contain):

   Customer ID

   Customer's name

   Customer gender

   Customer date of birth

   Driver's license number

   Auto policy number

   Note: you can also create an empty string variable by setting the value to `''` or a number variable with an initial value of 0

3. Regarding the variables you created, what assumptions did you make about the type of value that the variable might need to contain, or what other information might you need to better represent that variable? Make a note after the variable as an inline code comment, starting with #.

   Example:

   ```
   name = "Jeanette Her"  # I wrote this first name + last
          name, but maybe it should be the other way around? Or
          separate variables for each?
   ```

4. Create variables for:

   your own name

   the city and state where you were born or grew up

Lab 2

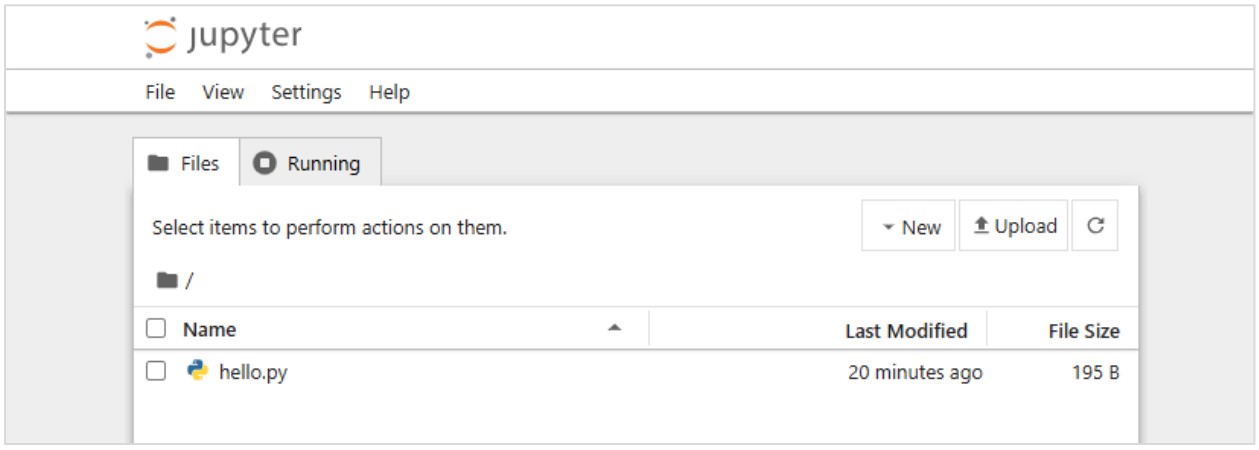
1. Go to https://flexiple.com/python/python-reserved-words. Answer the following questions **as comments** at the bottom of Ex1B_variable_examples.py:
   a) What is the full list of reserved words that can't be used for variable names?
   b) Pick 5 of these words and review the explanation for how it is used as a keyword in Python. Add these 5 definitions as # comments to your exercise document. Put ^^ around any terms that you are not familiar with.

2. Commit and push changes at the completion of these exercises.

BONUS (OPTIONAL)

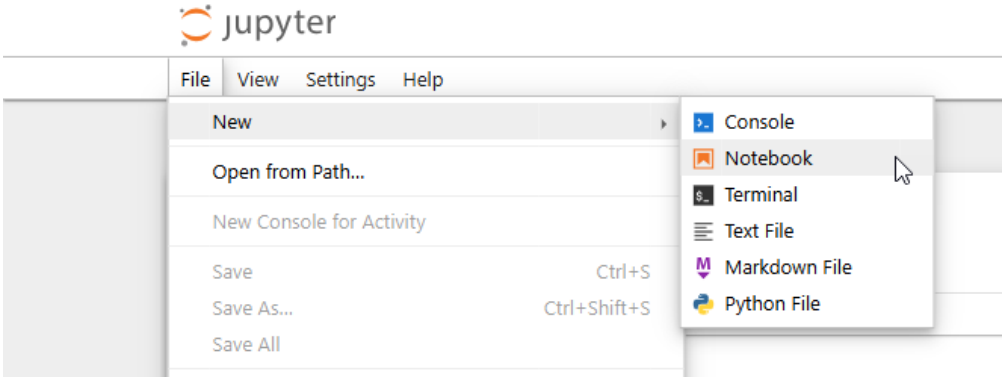
1. We'll work with Jupyter Notebooks more next week, but if you'd like to take a notebook for a test drive:
   a) Open Git Bash, and navigate to the DataAnalytics/week-05 folder.
   b) Once in week-05, enter the command **`jupyter notebook`**

      If the command returns invalid/not found, you may still need to install Jupyter Notebook. Using Git Bash, follow the pip install instructions found here: https://docs.jupyter.org/en/latest/install/notebook-classic.html
   c) Connection info will appear in Git Bash, and a browser window will open with the Jupyter Notebook dashboard. (You must leave Git Bash running while using Jupyter Notebook in the browser. If you look at the URL bar in your browser window, you can see that Jupyter Notebook is using a localhost connection, meaning it's running on your own local computer.)



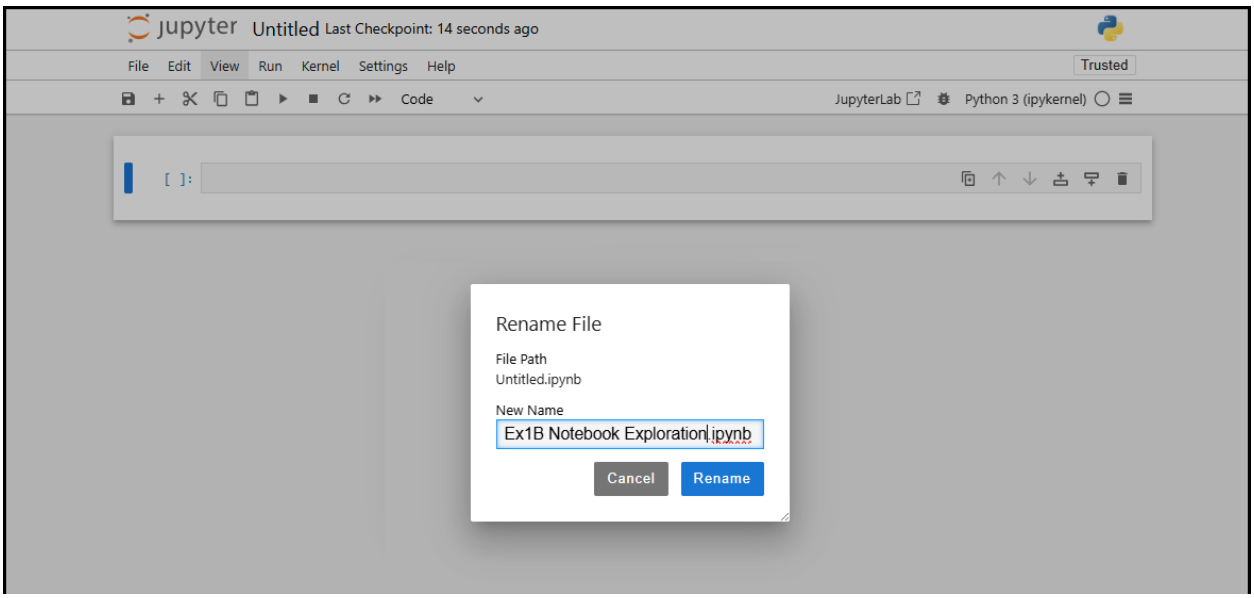
2. Once you have Jupyter Notebook open:

   a) Go to File > New > Notebook



   b) A pop-up will ask you to select a kernel. Keep the default option, and hit Select.

   c) Change the name of your new notebook at the top of page by clicking Untitled, and rename to **Ex1B_Notebook_Exploration.ipynb**



3. The first block in the notebook is a code block. You can write (or copy) code into the block displayed at the top of the notebook, and add new blocks above or below by clicking the icons ⍏ or ⍒

   a) In the top block, type `print('hello world')` and use play (▶) in the toolbar to run the code block. Notice how the result is displayed.

   b) In the next block, copy and paste into your notebook the list of variable_examples you just wrote. Is anything displayed when you run this block?

   c) In a third block, use print() to print one of the variables you added, then run the code block.

4. Save changes and close out of the notebook using File > Close & Shut Down Notebook. Then use File > Shut Down to exit Jupyter.

5. Remember, all your Jupyter Notebook work was done on localhost, so your files are only saved to your local computer. As a final step in Git Bash, you'll need to add, commit, and push your changes to week-05.

Module 2

Building Expressions

Exercise 2.A Make calculations with Python

Lab 1

In this exercise, you will use Visual Studio Code to create and then run a script.

Sample problem: How do you calculate the total due at a restaurant given the food cost, tax, and tip?

Formula: Total Due is determined by: Food Cost + Tax + Tip

Script:

```
# Define known values
food_cost = 79.25
tax = 6.54
tip = 12.00

# Calculate the unknown
total_due = food_cost + tax + tip

# Display the results
print("The total due is " + str(total_due))
```

To create the script:

1. Within **week-05**, create a new file named **Ex2A_tips.py**

2. Type the above script into the file.
   a) Remember what the `str()` function is for? Why is this being used here? Add your answer as a comment.

3. Use **File > Save** (or ctrl + S) to save the file.

4. Use the run icon (▷) to run the script. You will see the result in a Terminal window at the bottom.

5. Let's practice building a more complicated output for the print statement. Comment out the existing print statement by putting a # in front of it, then add the following new lines:

```
print("Food cost is " + str(food_cost) + " and tax is " + str(tax))
print("Tip is " + str(tip))
print("Total due is " + str(total_due))
```

6. Save your changes and re-run the script.

7. Do you notice how the tip amount doesn't have two digits to the right of the decimal point? Comment out the line of the print statement for the tip by putting a # in front of it, then add the following on a new line:

   ```
   print("Tip is " + format(tip, ".2f"))
   ```

   Save the script and re-run it. Looks better, right? Notice here that the `format()` function is used to format data as a string, so we do not need to also include the `str()` function with it.

8. Save and commit changes when finished.

Lab 2

Create a subfolder inside week-05 named **Ex2A_MathScripts**. All the scripts you write in the exercises below will be added to this folder.

For this exercise, collaborate with your peer group to find formulas and write your code. Note that VS Code runs the script that is currently open in the editor window. You may have multiple tabs in the editor window as you create new files. You can close tabs for files you have completed.

1. How do you calculate your net worth given your assets and debts?
   a) Start by brainstorming: What are "assets" that would need to be included in this calculation? What about "debts"?
   b) Discuss and figure out the formula and what the script would look like, making up example values as needed.
   c) Now create the script in a file named **net_worth.py**
   d) Your displayed output should be formulated as follows (3 print statements):
      `Your total assets are [number]`

```
Your total debts are [number]
Your net worth is [number]
```

e) Once your script is working, save and commit your changes.

2. How do you calculate the area of a rectangle?

   a) Say you have a rectangle that has dimensions corresponding to your birthday – the month number is one side and the day of the month is the other side. How would you calculate the area of this rectangle?

   b) Figure out the formula and what the script would look like, and create the script in a file named **area_of_rectangle.py**

   c) The displayed output should be formatted as follows:

```
Side A is [number]
Side B is [number]
The area of the rectangle is [number]
```

   d) Once your script is working, save and commit your changes.

3. How do you calculate the tip amount on a restaurant bill given the tip percentage?

   a) Figure out the formula and what the script would look like, making up example values as needed. (If you need inspiration, what was your approximate restaurant bill the last time you ate at a restaurant?)

   b) Create the script in a file named **tip_amount.py**

   c) The displayed output should be formatted as follows:

```
The tip on a $[number] restaurant bill is $[number]
```

   d) Once your script is working, save and commit your changes.

4. How do you calculate the area of a circle?

   a) The diameter of a given circle is the same as the day of your birthday (not the month, just the day). Figure out the formula, refresh your recollection of the difference between diameter and radius, and figure out what the script should look like.

   b) Create the script in a file named **area_of_circle.py**

   c) The displayed output should be formatted as follows:

```
The area of a circle with radius [number] is [number]
```

d) Once your script is working, save and commit your changes.

5. How long will it take a savings account worth `X` to double in value based on an interest rate of `IR`? (Hint: Look up the "rule of 72")

   a) Figure out the formula and what the script would look like, making up example values as needed.

   b) Create the script in a file named **rule_of_72.py**

   c) The displayed output should be formatted as follows:
   ```
   Your current savings is [number].
   At a [number]% interest rate, your savings account will be
   worth [number] in [number] years
   ```

   d) Show your doubled balance with 2 digits to the right of the decimal point by using `format(__, ".2f")` and show years with 1 digit to the right of the decimal. How can you do this using `format()`?

   e) There are a couple ways you might get the interest rate to display as a percentage. One option is to use the format function. In this case, instead of including the character *f* to assign a *fixed decimal* format, use the character *%* to assign the *percentage* format, e.g. `format(__, ".0%")`

   f) When you get the script working, commit your changes.

Lab 3:

The `input()` function allows you to create a variable that asks for the user running the script to input a value. For example:

```
bank_bal = input("What is your current bank balance? ")
print("Your balance is " + bank_bal)
```

When the above line of code is run, the terminal in VS Code will prompt:

`What is your current bank balance?`

You must enter a value in the terminal in order for the remainder of the script to continue to run. The value that you enter is what that variable will use in the program.

1. Choose one of the above exercises, and modify your script to create input options for the original variables. Customize the prompt within parentheses as appropriate, for example “What is the length of the first side?” or “How much do you want to tip?”

2. Run the updated script and experiment with different input values. Can you identify any possible pitfalls with using `input()`? Add your observations as comments in your script.

Lab 4:

Current versions of Python (3.6 and up) support formatted strings or **f-strings**. This is a simplified syntax for writing strings that can include variables and expressions within the string in curly braces.

Take an example we have already seen:

```
print("The total due is " + str(total_due))
```

This can be written as an f-string like this:

```
print(f"The total due is {total_due}")
```

Notice that an f-string is preceded by the character f, and the variable in braces is incorporated into the string without having to recast its type.

1. Choose one or more of the above exercises, and figure out how to re-write the print output as an f-string.

Make sure to confirm that you have pushed all changes at the completion of these exercises and that your week-05 folder is up to date in your DataAnalytics repository on GitHub.

Exercise 2.B More practical math scripts

Lab 1

Create a new subfolder in week-05 named **Ex2B_MoreMathScripts**. These problems will be a little more challenging. Collaborate with your peer group to figure out how to approach each problem, but try to code each script on your own. Utilize your group to review each other's code and share ideas for how you each solved the problem.

1. How do you convert a temperature from Fahrenheit to Celsius?

   Create the script in a file named **f_to_c.py**

2. How do you convert a temperature from Celsius to Fahrenheit?

   Create the script in a file named **c_to_f.py**

   This is a good opportunity to review each other's code in your peer group. Do they all look similar? The smaller the program is, the more likely the code is to be very similar.

3. Federal taxes are 23% of your salary every month. You make X amount of money. How much is withheld for taxes?

   Code the script in a file named **taxes.py**

   Make sure not to display partial cents in your results! Experiment with using either the round() function or format() method to get results to display to two decimal places.

4. How do you calculate the distance between coordinates (x1, y1) and (x2, y2)? Hint: You'll need to look up how to calculate a square root in Python, which may involve a function from the math module.

   Code the script for calculating this distance in a file named **distance.py**

5. You are going to tile a room whose dimensions are *length* by *width* feet. There are twelve tiles per box, each 1 foot by 1 foot. How many boxes of tiles do you need? You can only buy full boxes, not a partial box.

   You also want to buy at least 10% more tiles than you need in order to handle chips, breakage, and mess-ups. How many total boxes will you buy?

   Code the script in a file named **tiles.py**

6. There are X people going on a tour. Charter vans seat 15 passengers each. Vans cost $250 per day to rent (including the driver's pay). How many vans do you need? How much will it cost to rent vans? What is the cost if you split it per person?

   Code the script in a file named **rentals.py**

   Test your script with 38 tourists. Now do some separate calculations to check your work:

   a) How much money did your script say you had to charge per person?

   b) If you multiply that out, how much did you collect?

   c) How much were the vans?

   d) Why do you have leftover money?

7. Remember to commit and push your changes when you finish the exercises.

Module 3

Working with Data Types

Exercise 3.A Numeric conversion exercises

Lab 1

Create a new file directly in week-05 named **Ex3A_conversion_tests.py**.

1. Add a comment to the top using the following format:

```
# Description: This script tests various numeric
#              conversion techniques
# Author: Sam Q. Newprogrammer
```

2. Define the following variables:

```
a = "  101.1 "
b = '55'
c = "402 Stevens"
d = 'Number 5  '
```

3. For each variable above, perform the following transformations, creating a new variable for each.
   a) Run the script to test each new variable as you go.
   b) If a line of code produces an error, **comment out** that line but keep it in your script. Then note the error type as an inline comment.

4. Once you have all new variables created (and error-producing lines commented out), print the value of each variable and its type – e.g., `print(a, type(a))`

5. For each variable, what happens when you try the following? Add comments to your script to document each result.
   a) Cast as integer using `int()`
   b) Cast as float using `float()`
   c) For variable `a`, try casting into a float *then* integer, like this: `int(float(a))`

d) Use slicing to add just the numeric portion of the string to a new variable (remember, indexing always starts with 0!), and cast the number as an integer or string, whichever is appropriate

e) For variables `a` and `d`, use the `.strip()` method to remove the leading/trailing spaces, within a print statement to display each result.

6. Remember to commit and sync your changes when you finish.

Lab 2

1. Create a new file named **Ex3A_string_cleaning.py**.

2. A colleague has shared the following contact records, but the data is a mess, with inconsistent capitalization and currency symbols that need to be cleaned up before it can be used:

```
name_1 = "PRIYA SHARMA"
name_2 = "bob NGUYEN"
name_3 = "LaTonya Williams"
salary_1 = "$82,500"
salary_2 = "$74,000"
```

3. Use `.lower()` to convert all three names to lowercase, and print each result.

4. Use `.title()` to convert all three names to title case (first letter of each word capitalized), and print each result. (This is another useful method you can use alongside `.upper()` and `.lower()`.)

5. Use `.replace()` to remove the $ from both salary strings, and print each result. Add another print statement to test what data type these values are now. What would you need to do next to perform math on them?

6. Now chain `.replace()` and `int()` together in a single line to produce a usable integer from `salary_1`. Print the result and confirm its type using `type()`.

7. Remember to commit and sync your changes when you finish.

Exercise 3.B Creating collections

Lab 1

Create a new subfolder in week-05 named **Ex3B_SampleCollections**. The following exercises should be placed there.

1. Create a file with the name **movie_list.py**.

2. Create a list with the titles of your favorite movies (or movies you'd like to watch) – include at least 2, but no more than 10.

3. Use the len() function to print the descriptive statement:

   The list [list name] includes my top [length] favorite movies
   (Or: The list [list name] includes the [length] movies I'd like to watch)

   Then use another line of code to print the complete list.

4. Print a sorted list two ways (Note: make sure that your list items aren't already in alphabetical order to start with, or you won't notice any difference):

   a) Use the `sorted()` function to print a sorted list, then print the list again without using `sorted()`

      Example:
      ```
      print(sorted(listname))
      print(listname)
      ```

      What do you notice when you compare the two outputs?

   b) Use the `.sort()` method to sort the list, then print the list again, like this:

      ```
      listname.sort()
      print(listname)
      ```

      What do you notice when you compare the two outputs?

5. Think of at least one more movie to add to your list, and use the .append() method to add it. Then print the list again, also including an updated description statement.

6. Compare outputs with your group members. Did you achieve similar results?

Lab 2

1. Create a script named **candy_store.py**.

2. Start by creating two tuples: one that lists at least 3 types of candy that can come in fruit flavors, and another that lists at least 3 fruity flavors. (Feel free to get creative with your flavor ideas…)

3. Now create a new variable to store candy combinations as a set. Using the index of each tuple, add at least one combination of each candy and flavor to the new set – for example, putting together tuple1[0] and tuple2[1]

4. Create an output message that says, “Today’s candy options include:” and then prints the contents of the set.

5. Print the output multiple times. What do you notice about the order of the items as you repeat the output?

Lab 3

1. Create a file named **address_entry.py**, and in it define a dictionary named contact_info that includes the following keys and the sample values of your choice:

   name
   address
   city
   state
   zip

2. Print the address as properly formatted for mailing. Avoid using multiple print statements. Experiment with using a multi-line f-string (triple quotes), or use "\n" (new line) to break the address into multiple lines.

3. Remove the key:value pair for name.

4. Add a new variable for full_name and assign its value as a dictionary containing two key:value pairs. The first key:value pair should contain the key “first name” and a first name, and the second should contain the key “last name” and a last name.

5. Use the `.update()` method to add one more key:value pair to full_name, with the key “honorific” and the value set to Mr. / Ms. / Mx. / Dr. / Hon. / etc. as appropriate.

6. Use the `.update()` method to add full_name to contact_info.

7. Print the formatted address again, updating as needed to include the new dictionary items.

Remember to commit and push your changes when you finish the exercises.

Module 4

Conditional Logic & Flow Control

Exercise 4.A Using conditionals

Lab 1

Create a new folder in week-05 named **Ex4A_IfScripts**. These exercises should be placed there.

1. Create a new file named **if_basics.py**

2. Create two starting variables:

```
x = 100
y = 20
```

3. Express each of the following statements using an `if` block. Notice that each block uses the updated value of `x` set by the previous block:
   a) If x divided by y is 5, print “x divided by y is 5” and set the value of x to 1 – otherwise print “are the variables set up correctly?”
   b) If x times y is y, print “now x times y is y” and then set x to 10 – otherwise print “Whoops, x equals” + the value of x
   c) If x is less than y, print “x is less than y” and double the value of x – otherwise print “uh oh, x is not less than y”
   d) If x is greater than y, print “how is x greater than y??” otherwise print “x is NOT greater than y”
   e) Add a final print statement to say “The final value of x is __ and the final value of y is __” (displaying the actual values in place of the blanks)

Lab 2

1. In a file named **pay_rules.py**, create a script to calculate gross pay given the variables `pay_rate` and `hours_worked`. If the person works more than 40 hours, pay the overtime hours at 1.5 times the rate of regular hours.

2. When you are finished, review your script with a colleague. Are your algorithms similar? Do you believe each other's code will work?

3. Run your script several times with different values for pay_rate and hours_worked and confirm the output is right.

   a) Examples of good test data might be:

Pay Rate	**Hours Worked**	**Gross Pay**	**Reason**
12.50	20	250.00	Under 40 hours
25.50	40	1020.00	Exactly 40 hours
17.30	45	821.75	Over 40 hours

BONUS Lab (Optional)

1. In a file named **bonus_gregorian_calendar.py**, create a script to determine whether a given year is a leap year in the Gregorian calendar. You will need to do a little research to determine exactly what makes a year a leap year.

2. When you are finished, review your script with a colleague. Are your formulas similar? Do you believe each other's code will work?

3. Run it several times with different values for the year. Make sure to test the years 1900, 1950, 1999, 2000, 2001, and 20212.

DON'T FORGET: Commit and sync your changes when you finish these exercises.

Exercise 4.B Lookups and greetings

Lab 1

Create a new folder in week-05 named **Ex4B_MoreIfScripts**. The following exercises should be saved there.

1. Write a script named **dept_converter.py** that uses `if`/`elif`/`else` logic to determine and print department name based on a department code. Make sure to test your script with multiple codes.

1	Marketing
5	Human Resources
10	Accounting
12	Legal
18	IT
20	Customer Relations

2. Once your script is working, rewrite it using a `match`/`case` statement instead of `if`/`elif`/`else`. Save this version as **dept_converter_v2.py**

3. When you are done, compare scripts with a classmate. How did each of you approach solving this problem? Which solution do you think is most efficient? Which is easiest to read and understand? Are there any changes you would make to your own script based on seeing another example?

Lab 2

1. Create a script named **greeting.py**. Define a variable that contains the current hour (0-23). Display one of the greetings below based on the current hour:

Time	**Greeting**
until 10:00am	Good morning!
10:00am until 5:00pm	Good day!
5:00pm or later	Good evening!

Remember to test your script several times using different values for the hour.

2. Update your script to include an additional condition that will print “What are you doing up so late??” if the hour is between 11pm and 4am.

Lab 3

1. Create a script named **complex_taxes.py** that will calculate federal tax based on the values of annual gross income (a number) and a filing status (‘single’ or ‘joint’).

2. Start by copying your code for calculation of gross pay from the earlier lab (pay_rules.py) and include it here as part of your starting point. Remember, that code calculates *weekly* gross pay. Extend that calculation to estimate *annual* gross pay (how many weeks in a year?) and save it to a new variable.

3. Use a series of if statements to determine the appropriate tax rate.

   The tax table for single filers is:

Annual Income Range	Tax Rate
under 12,000	5%
12,000 - 24,999.99	10%
25,000 - 74,999.99	15%
75,000 and over	20%

   The tax table for joint filers is:

Annual Income Range	Tax Rate
under 12,000	0%
12,000 - 24,999.99	6%
25,000 - 74,999.99	11%
75,000 and over	20%

4. Use the tax rate to determine the tax withheld from the *weekly* gross pay.

5. Create separate print statements to print the relevant information determined by the above calculations. The output of your script might look something like this:

```
You worked __ hours this period.
```

```
Because you earn $___ per hour, your gross weekly pay is $475
Your filing status is ___
Your tax withholding for the week is $___
Your net pay is $___
```

6. To properly test the code, you will need to run it multiple times using different values for hours worked, pay rate, and filing status. Examine the tax tables and see if you can determine a set of good test values. Discuss with your small group – then run your tests!

Lab 4

1. Create a script named **min_max.py** that displays both the smallest and then the largest of three numbers.

   Name your variables `a`, `b`, and `c` and assign them values. Then use if/else statements to determine and display the answer.

   Be sure to test your script using an assortment of different values in your variables, so that you look at a variety of different number combinations.

Lab 5

1. Create a script named **show_major.py** that defines two variables for a student: `student_name` and `student_major`. The student_major variable will contain a code for the student's major (e.g. ENG).

2. Use the following table to create lookup logic to display the name of the major and location of the department's office based on the major code:

Major Code	Name of Major	Department Office
BIOL	Biology	Science Bldg, Room 310
CSCI	Computer Science	Sheppard Hall, Room 314
ENG	English	Kerr Hall, Room 201
HIST	History	Kerr Hall, Room 114
MKT	Marketing	Westly Hall, Room 310

What should your program do if a major code is not included in the table? Include an alternative to display `<unknown>` for the major name and nothing for location.

3. Run the script several times with different test values for student name and major code. (Make sure to include at least one test with a made-up major code not included in the table!)

DON'T FORGET: Commit your changes when you finish these exercises.

Exercise 4.C Loop de loop

Lab 1

Create a new subfolder in week-05 named **Ex4C_LoopScripts**. The following exercises should be placed there.

1. Create a script named **loop_love.py**.

2. Create a list of words that mean *great* such as *amazing*, *awesome*, *excellent*, etc.

3. Use a **`for`** loop to iterate through your list, and with each loop print out the phrase "Loops are ____!" filling in the blank with a word from the list.

4. When your loop has finished, print the additional phrase "I <3 loops"

Lab 2

1. Create a script named **savings_goal.py**.

2. Create a variable for your starting bank balance, another that sets your savings goal, and a third with your weekly savings amount.

3. Use a **`while`** loop to compare your bank balance to your savings goal, if you haven't met your goal yet, add the weekly savings amount to your bank balance. For each loop, print the statement, "This week my balance increased to ___." Once your savings goal is met, print the statement, "Goal met! My current balance is ___."

4. Try adding additional logic to your loop:
   a) If your balance is more than halfway to your goal, print the message, "Almost there! This week my balance is up to ___."
   b) If your balance is at least 75% of your goal, add a calculation to buy yourself a little treat. Print the message, "So close! After treating myself, my balance is up to ___."

5. Compare your for and while loops with one or more of your classmates. Did you achieve similar results? Did anyone get tripped up with getting the logic in the right order, or other sticky spots? Share with each other any parts that you may have gotten stuck on, and check out how others handled that in their code.

Lab 3

1. Create a script named **ranked_list.py**.

2. Create a list of at least 5 items using anything you like: favorite foods, pets, cities you'd like to visit, skills you want to develop, etc.

3. Use `enumerate()` with a `for` loop to print each item as a numbered list, starting at 1.

   Example:
   1. tacos
   2. ramen
   3. jerk chicken
   4. injera
   5. pierogi

4. Now add an `if` statement inside your loop: if the index is 1 (i.e., the first item), also print " <- top pick!" on the same line.

5. BONUS: Modify your loop to print the list in reverse order (still numbered 1 through 5) using `reversed()` around your list.

Lab 4

1. Create a script named **sales_performance.py**.

2. You have been given the following list of sales records. Each record is a tuple containing a salesperson's name, their region, and their total sales for the month:

```
sales_data = [
    ('Marcus Webb',    'East',  4250.00),
    ('Priya Sharma',   'West',  5875.50),
```

```
    ('DeShawn Carter', 'East',  3100.75),
    ('LaTonya Rivers', 'South', 6420.00),
    ('Bob Nguyen',     'West',  4980.25),
]
```

3. Use a `for` loop to unpack each tuple directly in the loop statement, and print a summary line for each record that looks like this:

   Marcus Webb (East): $4,250.00
   Priya Sharma (West): $5,875.50
   ...

4. Add a conditional inside your loop: if a salesperson's total is greater than $5,000, also print " ^ Top performer!" below their summary line.

5. BONUS: Add a variable before the loop to track total sales across all employees, and print the overall total after the loop finishes.

DON’T FORGET: Commit and sync your changes when you finish these exercises.

Exercise 4.D Bonus Project: Number Guesser

GAME TIME! (Optional)

Create a script in week-05 named **Ex4D_Bonus_number_guesser.py**. The following exercise should be placed there.

1. Write a simple game that you can run in a terminal that generates a random integer and makes the user guess that integer using `input()` until they guess correctly.

2. Start by displaying the possible range of numbers to get the user started guessing.
   a) Note: While Python does include a `random` module that can be used to generate a random number, that's not what you should use for this exercise.
   b) Instead, use the concepts that we've discussed in the previous modules to create your own pool of numbers and figure out how to randomize its order.
   c) Hint: You will need to use at least 2 collection data types in your solution.

3. Prompt the user for their guess.

4. Show "higher" or "lower" hints, and have the user continue to guess until they get it right.

5. Bonus add-ons:
   a) Keep track of the number of guesses
   b) Collect all the guessed numbers and print them at the end
   c) If the user takes less than 5 guesses, print that they're awesome
   d) Makes the program safe for non-numeric input (meaning, how does the program respond if the user types something that's not a number?)

When you are done, make sure to confirm that you have committed and synced all exercise files and that your week-05 folder is up to date in your DataAnalytics repository on GitHub.

End of Week 5 Lab Workbook